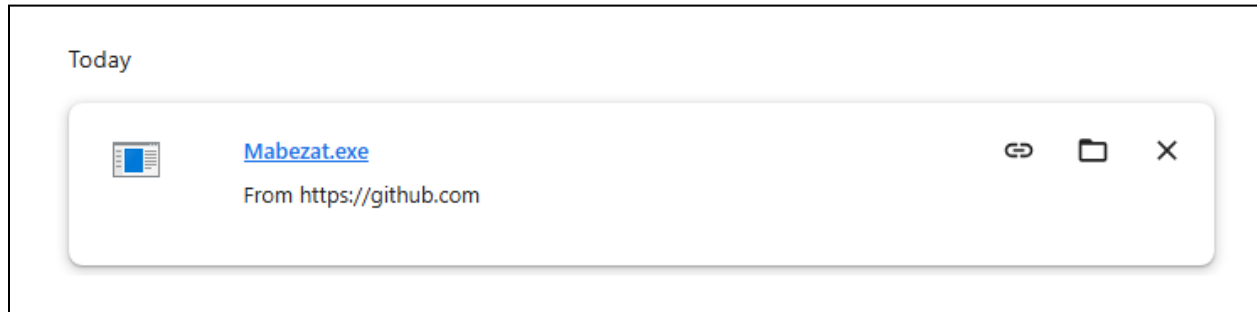


Static Analysis of Malware

Malware - <https://github.com/Da2dalus/The-MALWARE-Repo/tree/master/Virus/Mabezat>

Download a .exe malware sample from the website (Do not download ransomware, worms, and rogues.)



Your task is to perform static analysis on this file and extract as much useful information as possible without executing the malware.

File Information: Analyze the file properties

For static analysis, I have downloaded the Mabezat.exe malware from the respective repository.

general		
compiler-stamp	0x48025179	Sun Apr 13 18:31:21 2008 UTC
size-of-optional-header	0x00E0	224 bytes
signature	0x00004550	PE00
machine	0x014C	Intel-386
sections-count	0x0003	3
pointer-symbol-table	0x00000000	0x00000000
number-of-symbols	0x00000000	0x00000000

By analyzing the File Headers, we can see that the

- File is created on 13 April 2008
- Size of the file is 224 Bytes
- Signature is PE00 (This confirms that the file is a Portable Executable (PE))
- Number of section is 3

```

FLARE-VM 14-10-2025 17:26:20.72
C:\Users\hrp\Desktop>file Mabezat.exe
Mabezat.exe: PE32 executable (GUI) Intel 80386, for MS Windows

FLARE-VM 14-10-2025 17:26:36.45
C:\Users\hrp\Desktop>certutil -hashfile "C:\Users\hrp\Desktop\Mabezat.exe" MD5
MD5 hash of C:\Users\hrp\Desktop\Mabezat.exe:
de8d08a3018dfe8fd04ed525d30bb612
CertUtil: -hashfile command completed successfully.

FLARE-VM 14-10-2025 17:26:39.68
C:\Users\hrp\Desktop>certutil -hashfile "C:\Users\hrp\Desktop\Mabezat.exe" SHA256
SHA256 hash of C:\Users\hrp\Desktop\Mabezat.exe:
2ae0c4a5f1fedf964e2f8a486bf0ee5d1816aac30c889458a9ac113d13b50ceb
CertUtil: -hashfile command completed successfully.

```

the **file** command is a standard utility in Unix-like operating systems (Linux, macOS, etc.) that is used to **determine the type of a file**. Here, we can see that the file is a **Portable Executable (PE)**, meaning it is a standard executable format used by Windows operating systems.

We are also taking the SHA256 and MD5 of the file which resulted in

SHA256 - 2ae0c4a5f1fedf964e2f8a486bf0ee5d1816aac30c889458a9ac113d13b50ceb

2ae0c4a5f1fedf964e2f8a486bf0ee5d1816aac30c889458a9ac113d13b50ceb

58/71 security vendors flagged this file as malicious

Community Score: 58/71

Size: 141.00 KB

Last Analysis Date: 1 month ago

File type: EXE

Popular threat label: **worms.mabezat**

Threat categories: worm, virus

Family labels: mabezat

Security vendors' analysis:

Vendor	Detection	Vendor	Detection
AhnLab-V3	Virus:Win.Mabezat.144384	Alibaba	Virus/Win32/Mabezat.9602208d
AllCloud	Worm	ALYac	Win32.Worm.Mabezat.G
Antiy-AVL	Worm/Win32.Mabezat.b	Arcabit	Win32.Worm.Mabezat.G
Arctic Wolf	Unsafe	Avast	Win32:Mabezat [Wrm]
AVG	Win32:Mabezat [Wrm]	Avira (no cloud)	W32/Mabezat
BitDefender	Win32.Worm.Mabezat.G	Bkav Pro	W32.Pharaoh.PE
ClamAV	Win.Trojan.Mabezat-3	CrowdStrike Falcon	Win/malicious_confidence_100% (W)
CTX	Exe.virus.mabezat	DeepInstinct	MALICIOUS
DrWeb	Win32.HLLW.Tazebama	Emsisoft	Win32.Worm.Mabezat.G (B)

Md5 - de8d08a3018dfe8fd04ed525d30bb612

58/71 security vendors flagged this file as malicious

2ae0c45f1fed964e2f8a486bf0ee5d1816aac30c889458a9ac113d13b50ceb
mobisync.exe
Size: 141.00 KB
Last Analysis Date: 1 month ago

Community Score: 58/71

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: worm.mabezat
Threat categories: worm, virus
Family labels: mabezat

Security vendors' analysis

Vendor	Detection	Vendor	Detection
AhnLab-V3	Virus/Win.Mabezat.144384	Alibaba	Virus/Win32/Mabezat.9602208d
AliCloud	Worm	ALYac	Win32.Worm.Mabezat.G
Antiy-AVL	Worm/Win32.Mabezat.b	Arcabit	Win32.Worm.Mabezat.G
Arctic Wolf	Unsafe	Avast	Win32:Mabezat (Wrm)
AVG	Win32:Mabezat (Wrm)	Avira (no cloud)	W32/Mabezat
BitDefender	Win32.Worm.Mabezat.G	Bitav Pro	W32.Pharaoh.PE
ClamAV	Win.Trojan.Mabezat-3	CrowdStrike Falcon	Win/malicious_confidence_100% (W)
CTX	Exe.virus.mabezat	DeepInstinct	MALICIOUS
DnWeb	Win32.HLLW.Tazebama	Emsisoft	Win32.Worm.Mabezat.G (B)

From the results of [VirusTotal.com](https://www.virustotal.com) , we could confirm that the sample file is suspicious and its a malware

String Analysis:

```
FLARE-VM 14-10-2025 17:27:15.95
C:\Users\hrp\Desktop>strings.exe -n 5 -accepteula Mabezat.exe > strings_sys.txt

Strings v2.54 - Search for ANSI and Unicode strings in binary images.
Copyright (C) 1999-2021 Mark Russinovich
Sysinternals - www.sysinternals.com
```

String extraction was performed using the Sysinternals **strings.exe** tool, filtering for strings of 5 or more characters and redirecting the output to a text file strings_sys.txt). A custom **Python program** was then used to parse and categorize the extracted strings, allowing for efficient identification of key indicators.

On Observing the output of the python program we could see that there are some information regarding the Domain, IP, ExeName & so on.

```
=== Categorization summary ===
Domain      : 28
Email       : 0
ExeName     : 19
IP          : 2
RegKey      : 0
URL         : 0
Unclassified: 591
UnixPath    : 29
WinPath     : 0
```

PE Header Analysis (for PE files):

The sample contains three main sections with the following details:

property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.data	.rsrc
footprint > sha256	66E79FE8BDC675311CDA212...	794217D07EB9999B48979BC...	854173C8BA4B81EC452717E...
entropy	6.415	2.285	5.532
file-ratio (99.29%)	60.64 %	0.35 %	38.30 %
raw-address (begin)	0x00000400	0x00015A00	0x00015C00
raw-address (end)	0x00015A00	0x00015C00	0x00023400
raw-size (143360 bytes)	0x00015600 (87552 bytes)	0x00000200 (512 bytes)	0x0000D800 (55296 bytes)
virtual-address	0x00001000	0x00017000	0x00018000
virtual-size (142908 bytes)	0x00015478 (87160 bytes)	0x000003A4 (932 bytes)	0x0000D620 (54816 bytes)

Observations:

- **.text:** Contains the executable code. Its large size (87.5 KB Raw) is expected for the main code section.
- **.data:** Contains initialized data. The **Virtual Size (932 bytes)** is larger than the **Raw Size (512 bytes)**. This difference often indicates that the section contains uninitialized data or is expanded during runtime.
- **.rsrc:** Contains resources like icons, manifests, or embedded data.

Entropy analysis helps detect packing or encryption. An entropy value close to **8.0** indicates random-looking data, which is typical for encrypted or compressed content. The entropy values **do not strongly indicate a heavily packed or encrypted file** (like UPX or MPRESS)

because no section is close to 8.0. However, the **.text** section's moderate value (6.415) suggests some anti-analysis measures are in place, potentially light obfuscation or a custom packer.

Extract information about imported and exported functions.

imports (212)	flag (21)	first-thunk-original (INT)	first-thunk (IAT)	hint	group (14)	technique (9)	type (4)	ordinal (1)	library (0)
OpenThreadToken	×	0x00015670	0x77F472BC	431 (0x01AF)	security	T1134 Access Token Manipulation	implicit	-	ADVAPI32.dll
OpenProcessToken	×	0x0001565C	0x77F4797B	426 (0x01AA)	security	T1134 Access Token Manipulation	implicit	-	ADVAPI32.dll
LoadCursorA	-	0x00015D64	0x7E3AD33E	439 (0x01B7)	resource	-	implicit	-	USER32.dll
LoadIconA	-	0x00015D72	0x7E3AE8F6	443 (0x01BB)	resource	-	implicit	-	USER32.dll
LoadStringA	-	0x00015FFA	0x7E3AC908	456 (0x01C8)	resource	-	implicit	-	USER32.dll
LoadStringW	-	0x00016008	0x7E399E36	457 (0x01C9)	resource	-	implicit	-	USER32.dll
RegQueryValueExW	-	0x0001572C	0x77F46EF	495 (0x01EF)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegQueryValueExA	-	0x00015718	0x77F47AAB	494 (0x01EE)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegEnumKeyW	×	0x0001574E	0x77F4D5D4	474 (0x01DA)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegEnumKeyA	×	0x00015740	0x77F53398	471 (0x01D7)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegOpenKeyExW	-	0x00015708	0x77F46A9F	485 (0x01E5)	registry	-	implicit	-	ADVAPI32.dll
RegOpenKeyExA	-	0x000156F8	0x77F47842	484 (0x01E4)	registry	-	implicit	-	ADVAPI32.dll
RegCreateKeyExW	×	0x000156C6	0x77F4775C	463 (0x01CF)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegCreateKeyExA	×	0x000156B4	0x77F4E9E4	462 (0x01CE)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegSetValueExW	×	0x000156A2	0x77F4D757	508 (0x01FC)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegSetValueExA	×	0x00015690	0x77F4EAD7	507 (0x01FB)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegCloseKey	-	0x00015682	0x77F46C17	458 (0x01CA)	registry	-	implicit	-	ADVAPI32.dll
GetUserNameW	-	0x000156E8	0x77F5494D	292 (0x0124)	reconnaissance	T1033 System Owner/User Discovery	implicit	-	ADVAPI32.dll
GetUserNameA	-	0x000156D8	0x77F554A4	291 (0x0123)	reconnaissance	T1033 System Owner/User Discovery	implicit	-	ADVAPI32.dll
GetStartupInfoA	-	0x00015B94	0x7C801EF2	430 (0x01AE)	reconnaissance	-	implicit	-	KERNEL32.dll
GetTickCount	-	0x00015B10	0x7C80932E	468 (0x01D4)	reconnaissance	T1124 System Time Discovery	implicit	-	KERNEL32.dll
GetVersionExA	-	0x000158E4	0x7C812B6E	478 (0x01DE)	reconnaissance	-	implicit	-	KERNEL32.dll
QueryPerformanceCounter	-	0x00015990	0x7C80A4B7	660 (0x0294)	reconnaissance	-	implicit	-	KERNEL32.dll
GetCurrentProcessId	×	0x000159AA	0x7C8099B0	316 (0x013C)	reconnaissance	T1057 Process Discovery	implicit	-	KERNEL32.dll
GetUserDefaultLCID	-	0x00015A80	0x7C809FA0	472 (0x01D8)	reconnaissance	-	implicit	-	KERNEL32.dll
GetDateFormatA	-	0x00015B08	0x7C836206	319 (0x013F)	reconnaissance	-	implicit	-	KERNEL32.dll
GetDateFormatW	-	0x00015B1A	0x7C83378D	320 (0x0140)	reconnaissance	-	implicit	-	KERNEL32.dll
GetTimeFormatA	-	0x00015B2C	0x7C836345	469 (0x01D5)	reconnaissance	-	implicit	-	KERNEL32.dll
GetTimeFormatW	-	0x00015B3E	0x7C833FEB	470 (0x01D6)	reconnaissance	-	implicit	-	KERNEL32.dll
GetSystemMetrics	-	0x00015E4E	0x7E398F9C	349 (0x015D)	reconnaissance	-	implicit	-	USER32.dll
LocalReAlloc	-	0x00015A5A	0x7C830917	593 (0x0251)	memory	-	implicit	-	KERNEL32.dll
LocalAlloc	-	0x00015A40	0x7C809A1D	586 (0x024A)	memory	-	implicit	-	KERNEL32.dll
LocalFree	-	0x00015A4E	0x7C8099BF	590 (0x024E)	memory	-	implicit	-	KERNEL32.dll
CoTaskMemFree	-	0x00016186	0x774BD044	100 (0x0064)	memory	-	implicit	-	ole32.dll
FileTimeToSystemTime	-	0x000158F4	0x7C80E87C	195 (0x00C3)	file	-	implicit	-	KERNEL32.dll
FileTimeToLocalFileTime	-	0x0001590C	0x7C80E8F6	194 (0x00C2)	file	-	implicit	-	KERNEL32.dll
GetSystemTimeAsFileTime	-	0x000159C0	0x7C8017E9	448 (0x01C0)	file	T1124 System Time Discovery	implicit	-	KERNEL32.dll
GetCurrentThreadId	×	0x00015782	0x7C8097B8	318 (0x013E)	execution	T1057 Process Discovery	implicit	-	KERNEL32.dll
CreateThread	-	0x00015878	0x7C8106C7	108 (0x006C)	execution	-	implicit	-	KERNEL32.dll
GetCurrentProcess	×	0x00015888	0x7C80DEB5	315 (0x013B)	execution	T1057 Process Discovery	implicit	-	KERNEL32.dll
SetEnvironmentVariableW	×	0x0001589C	0x7C81024E	770 (0x0302)	execution	-	implicit	-	KERNEL32.dll
GetCurrentThread	×	0x00015886	0x7C80997B	317 (0x013D)	execution	-	implicit	-	KERNEL32.dll

The Mabezat.exe sample imports functions primarily from ADVAPI32.dll, KERNEL32.dll, and USER32.dll. The sheer volume and nature of the imports strongly suggest the malware is designed for system persistence, reconnaissance, and execution control.

Key Imported DLLs:

- **ADVAPI32.dll:** Used for advanced Windows services, including **Registry** manipulation and service control.
- **KERNEL32.dll:** Core system functions, including process, thread, memory management, and file operations.
- **USER32.dll:** User interface functions, including desktop and window management.

Obfuscation/Encryption Detection:

a. Identify any signs of code obfuscation or encryption within the binary.

Yes, there are strong signs of obfuscation and anti-analysis techniques in **Mabezat.exe**:

property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.data	.rsrc
footprint > sha256	66E79FE8BDC675311CDA212...	794217D07EB9999B48979BC...	854173C8BA4B81EC452717E...
entropy	6.415	2.285	5.532
file-ratio (99.29%)	60.64 %	0.35 %	38.30 %
raw-address (begin)	0x0000400	0x00015A00	0x00015C00
raw-address (end)	0x00015A00	0x00015C00	0x00023400
raw-size (143360 bytes)	0x00015600 (87552 bytes)	0x00000200 (512 bytes)	0x0000D800 (55296 bytes)
virtual-address	0x00001000	0x00017000	0x00018000
virtual-size (142908 bytes)	0x00015478 (87160 bytes)	0x000003A4 (932 bytes)	0x0000D620 (54816 bytes)

- **Dynamic API Resolution:** The sample imports both LoadLibrary and GetProcAddress. This is a classic evasion technique where the malware avoids listing critical functions (like network or file I/O functions) directly in its Import Address Table (IAT). Instead, it resolves and calls these functions at runtime, complicating basic static analysis tools.
- **Moderate Section Entropy:** The .text section (the primary code section) has an entropy value of 6.415. While this is not high enough (values near 7.5-8.0 are definitive for heavy packing), it is above the typical range for plain compiled code and suggests the use of light obfuscation or custom encoding within the executable instructions.
- **Suspicious API Calls:** The presence of function imports like CreateThread and potential memory manipulation functions (e.g., searching for VirtualAlloc, WriteProcessMemory per best practice), further supports the idea that the code is designed to unpack, inject, or run malicious code in a hidden way.

-	UnhandledExceptionFilter
-	SetUnhandledExceptionFilter
-	FreeLibrary
-	GetProcAddress
T1106 Execution through API	LoadLibrary
T1106 Execution through API	LoadLibrary
-	GetModuleHandle
-	CoFreeUnusedLibraries
-	LoadRegTypeLib
-	GetLastError
-	SetLastError
-	FormatMessage

b. Extract any possible encoded/encrypted strings or resources.

- The .rsrc (resources) section has a moderate entropy of 5.532. This section often contains compressed or encoded data, potentially including an embedded payload or configuration that would be decrypted during execution.

Section Entropy Analysis

Entropy analysis helps detect packing or encryption. An entropy value close to **8.0** indicates random-looking data, which is typical for encrypted or compressed content.

property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.data	.rsrc
footprint > sha256	66E79FE8BDC675311CDA212...	794217D07EB9999B48979BC...	854173C8BA4B81EC452717E...
entropy	6.415	2.285	5.532
file-ratio (99.29%)	60.64 %	0.35 %	38.30 %
raw-address (begin)	0x0000400	0x00015A00	0x00015C00
raw-address (end)	0x00015A00	0x00015C00	0x00023400
raw-size (143360 bytes)	0x00015600 (87552 bytes)	0x00000200 (512 bytes)	0x0000D800 (55296 bytes)
virtual-address	0x00001000	0x00017000	0x00018000
virtual-size (142908 bytes)	0x00015478 (87160 bytes)	0x000003A4 (932 bytes)	0x0000D620 (54816 bytes)

Observations:

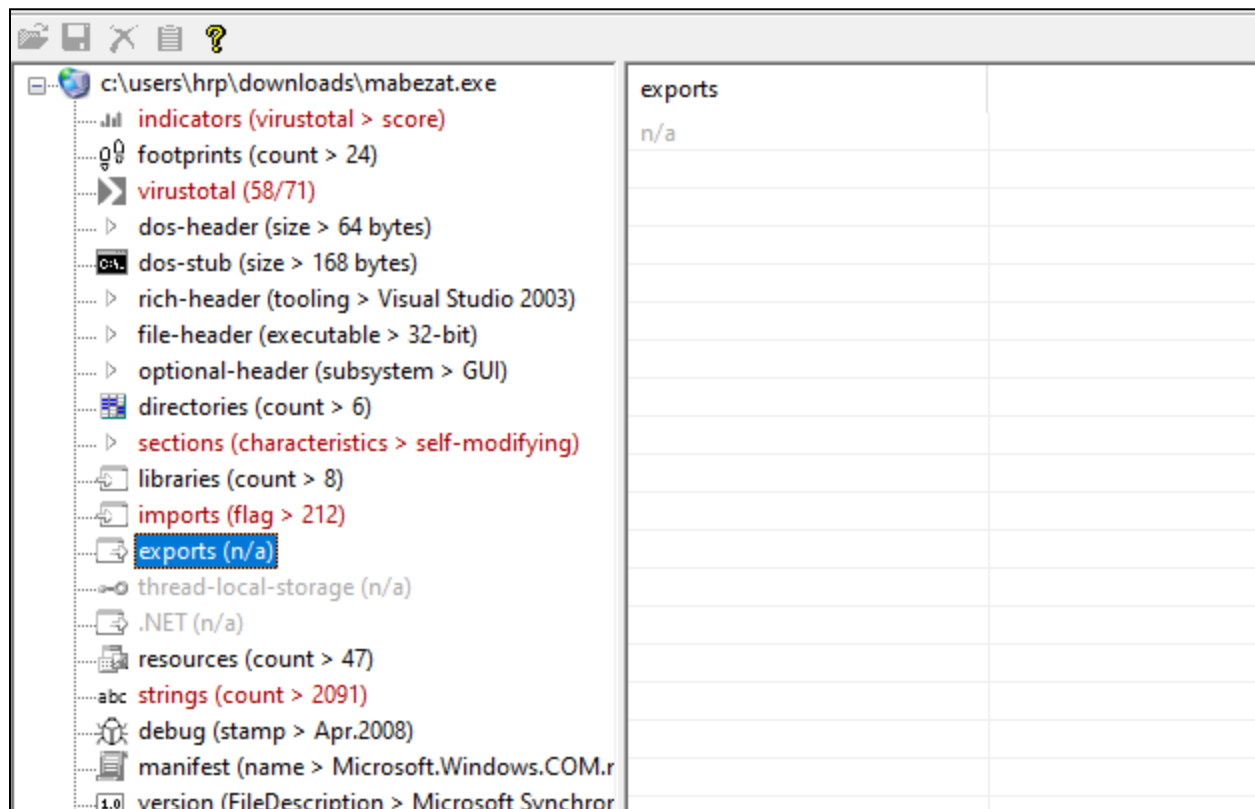
Section Name	Entropy Value	Interpretation
.text	6.415	Moderate Entropy. This value is higher than typical for uncompiled code (usually 5.0-6.0). It suggests some level of data obfuscation or non-standard compilation, but is not definitive proof of heavy packing (which would be near 7.5-8.0).
.data	2.285	Low Entropy. This is typical for data sections containing simple initialized variables and pointers.
.rsrc	5.532	Moderate Entropy. This is a common entropy range for resource sections, which often contain compressed or complex resource data (e.g., icons, images).

Detect any unusual API calls

API Call	Library	Malicious Purpose
RegOpenKeyEx / RegSetValueEx / RegCreateKeyEx	ADVAPI32	Registry Manipulation for persistence, configuration, or data storage.
CreateThread	KERNEL32	Creates an additional thread of execution, often to hide malicious activity or perform tasks concurrently.
SetEnvironmentVariable / GetEnvironmentVariable	KERNEL32	Used for storing configuration or paths, potentially for persistence or dynamic payload loading.
OpenProcessToken / LoadLibrary	KERNEL32	Access Token Manipulation (privilege escalation) and Dynamic Loading (evasion).
GetSystemTimeAsFileTime / GetTimeZoneInformation	KERNEL32	System Time Discovery, potentially for anti-debugging or time-based execution.

Export Table Review

There is no result that are visible in the export table



Section Anomalies Detection

property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.data	.rsrc
footprint > sha256	66E79FE8BDC675311CDA212...	794217D07EB9999B48979BC...	854173C8BA4B81EC452717E...
entropy	6.415	2.285	5.532
file-ratio (99.29%)	60.64 %	0.35 %	38.30 %
raw-address (begin)	0x00000400	0x00015A00	0x00015C00
raw-address (end)	0x00015A00	0x00015C00	0x00023400
raw-size (143360 bytes)	0x00015600 (87552 bytes)	0x00000200 (512 bytes)	0x0000D800 (55296 bytes)
virtual-address	0x00001000	0x00017000	0x00018000
virtual-size (142908 bytes)	0x00015478 (87160 bytes)	0x000003A4 (932 bytes)	0x0000D620 (54816 bytes)

- **RWE Permissions:** Highly Suspected. The presence of dynamic API calls (LoadLibrary, GetProcAddress) strongly suggests the malware intends to allocate Read/Write/Execute (RWE) memory at runtime for unpacking and injecting code.
- **Section Alignment Anomaly:** Yes. The .data section has a Virtual Size (932 bytes) that is larger than its Raw Size (512 bytes). This indicates uninitialized data (BSS) that expands when loaded into memory.
- **Non-standard Section Names:** None. The file uses standard names (.text, .data, .rsrc), meaning it avoids easy detection by basic packer signature checks.

Suspicious API Pattern Identification

imports (212)	flag (21)	first-thunk-original (INT)	first-thunk (IAT)	hint	group (14)	technique (9)	type (4)	ordinal (1)	library (0)
OpenThreadToken	x	0x00015670	0x77F472BC	431 (0x01AF)	security	T1134 Access Token Manipulation	implicit	-	ADVAPI32.dll
OpenProcessToken	x	0x0001565C	0x77F4797B	426 (0x01AA)	security	T1134 Access Token Manipulation	implicit	-	ADVAPI32.dll
LoadCursorA	-	0x00015D64	0x7EAD33E	439 (0x0187)	resource	-	implicit	-	USER32.dll
LoadIconA	-	0x00015D72	0x7E3AE8F6	443 (0x018B)	resource	-	implicit	-	USER32.dll
LoadStringA	-	0x00015EFA	0x7E3AC908	456 (0x01C8)	resource	-	implicit	-	USER32.dll
LoadStringW	-	0x00016008	0x7E399E36	457 (0x01C9)	resource	-	implicit	-	USER32.dll
RegQueryValueExW	-	0x0001572C	0x77F46FEF	495 (0x01EF)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegQueryValueExA	-	0x00015718	0x77F47AAB	494 (0x01EE)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegEnumKeyW	x	0x0001574E	0x77F4D5D4	474 (0x01DA)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegEnumKeyA	x	0x00015740	0x77F53398	471 (0x01D7)	registry	T1012 Query Registry	implicit	-	ADVAPI32.dll
RegOpenKeyExW	-	0x00015708	0x77F46A9F	483 (0x01E5)	registry	-	implicit	-	ADVAPI32.dll
RegOpenKeyExA	-	0x000156F8	0x77F47842	484 (0x01E4)	registry	-	implicit	-	ADVAPI32.dll
RegCreateKeyExW	x	0x000156C6	0x77F4775C	463 (0x01CF)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegCreateKeyExA	x	0x000156B4	0x77F4E9E4	462 (0x01CE)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegSetValueExW	x	0x000156A2	0x77F4D757	508 (0x01FC)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegSetValueExA	x	0x00015690	0x77F4EAD7	507 (0x01FB)	registry	T1112 Modify Registry	implicit	-	ADVAPI32.dll
RegCloseKey	-	0x00015682	0x77F46C17	458 (0x01CA)	registry	-	implicit	-	ADVAPI32.dll
GetUserNameW	-	0x000156E8	0x77F5494D	292 (0x0124)	reconnaissance	T1033 System Owner/User Discovery	implicit	-	ADVAPI32.dll
GetUserNameA	-	0x000156D8	0x77F554A4	291 (0x0123)	reconnaissance	T1033 System Owner/User Discovery	implicit	-	ADVAPI32.dll
GetStartupInfoA	-	0x00015B94	0x7C801EF2	430 (0x01AE)	reconnaissance	-	implicit	-	KERNEL32.dll
GetTickCount	-	0x00015B10	0x7C80932E	468 (0x01D4)	reconnaissance	T1124 System Time Discovery	implicit	-	KERNEL32.dll
GetVersionExA	-	0x00015B64	0x7C812B6E	478 (0x01DE)	reconnaissance	-	implicit	-	KERNEL32.dll
QueryPerformanceCounter	-	0x00015990	0x7C80A4B7	660 (0x0294)	reconnaissance	-	implicit	-	KERNEL32.dll
GetCurrentProcessId	x	0x000159AA	0x7C8099B0	316 (0x013C)	reconnaissance	T1057 Process Discovery	implicit	-	KERNEL32.dll
GetUserDefaultCLID	-	0x00015A80	0x7C809FA0	472 (0x01D8)	reconnaissance	-	implicit	-	KERNEL32.dll
GetDateFormatA	-	0x00015B08	0x7C836206	319 (0x013F)	reconnaissance	-	implicit	-	KERNEL32.dll
GetDateFormatW	-	0x00015B1A	0x7C8337BD	320 (0x0140)	reconnaissance	-	implicit	-	KERNEL32.dll
GetTimeFormatA	-	0x00015B2C	0x7C836345	469 (0x01D5)	reconnaissance	-	implicit	-	KERNEL32.dll
GetTimeFormatW	-	0x00015B3E	0x7C833FE8	470 (0x01D6)	reconnaissance	-	implicit	-	KERNEL32.dll
GetSystemMetrics	-	0x00015E4E	0x7E398F9C	349 (0x015D)	reconnaissance	-	implicit	-	USER32.dll
LocalReAlloc	-	0x00015A5A	0x7C830917	593 (0x0251)	memory	-	implicit	-	KERNEL32.dll
LocalAlloc	-	0x00015A40	0x7C809A1D	586 (0x024A)	memory	-	implicit	-	KERNEL32.dll
LocalFree	-	0x00015A4E	0x7C8099BF	590 (0x024E)	memory	-	implicit	-	KERNEL32.dll
CoTaskMemFree	-	0x00016186	0x774BD044	100 (0x0064)	memory	-	implicit	-	ole32.dll
FileTimeToSystemTime	-	0x00015BFA	0x7C80E87C	195 (0x00C3)	file	-	implicit	-	KERNEL32.dll
FileTimeToLocalFileTime	-	0x000159DC	0x7C80E8F6	194 (0x00C2)	file	-	implicit	-	KERNEL32.dll
GetSystemTimeAsFileTime	-	0x000159C0	0x7C8017E9	448 (0x01C0)	file	T1124 System Time Discovery	implicit	-	KERNEL32.dll
GetCurrentThreadId	x	0x00015782	0x7C8097B8	318 (0x013E)	execution	T1057 Process Discovery	implicit	-	KERNEL32.dll
CreateThread	-	0x00015878	0x7C8106C7	108 (0x006C)	execution	-	implicit	-	KERNEL32.dll
GetCurrentProcess	x	0x00015888	0x7C80DE85	315 (0x013B)	execution	T1057 Process Discovery	implicit	-	KERNEL32.dll
SetEnvironmentVariableW	x	0x0001589C	0x7C81024E	770 (0x0302)	execution	-	implicit	-	KERNEL32.dll
GetCurrentThread	x	0x000158B6	0x7C80997B	317 (0x013D)	execution	-	implicit	-	KERNEL32.dll

The imported functions reveal the following malicious patterns:

- **Persistence & Configuration:** Uses RegOpenKeyEx and RegSetValueEx (Registry access) combined with SetEnvironmentVariable to ensure startup and store settings.
- **Execution Evasion:** Uses LoadLibrary and GetProcAddress for dynamic API resolution, loading critical functions at runtime to bypass static detection. This is often followed by CreateThread for hidden execution.
- **System Reconnaissance:** Gathers host information using GetUserNameA and GetComputerNameA, essential steps before launching the main payload.
- **Privilege Access:** Imports OpenProcessToken, indicating an attempt to manipulate process security tokens, a precursor to privilege escalation.

Timestamp and Compiler Information Analysis

general		
compiler-stamp	0x48025179	Sun Apr 13 18:31:21 2008 UTC
size-of-optional-header	0x00E0	224 bytes
signature	0x00004550	PE00
machine	0x014C	Intel-386
sections-count	0x0003	3
pointer-symbol-table	0x00000000	0x00000000
number-of-symbols	0x00000000	0x00000000

- **Timestamp Age:** The timestamp dates the file compilation to **2008**. While possible for older malware, the date is often **spoofed** (modified by the attacker) to mimic an older, established binary or to throw off time-based analysis.
- **Compiler Signature:** The architecture is **32-bit (i386)**. The specific compiler is not explicitly listed in the screenshots but the general PE characteristics are typical of a standard Windows development environment.
- **Implication:** No definitive anomaly (like a future date or zeroed timestamp) is present, but the **old 2008 date is suspicious** and should be treated as unreliable metadata.